Development of an LLM-Driven Framework for ARG Detection Based on a Genomic Database

By

011221373	Md Habibulla Misba
0112230359	Kazi Neyamul Hasan
0112230362	Rakibul Hassan
0112230367	Abdullah Al Noman
0112230889	Mahathir Mohammad

Submitted in partial fulfilment of the requirements
of the degree of Bachelor of Science in Computer Science and Engineering

February 25, 2026

Department of Computer Science and Engineering
United International University

Abstract

The abstract should contain a summary of the work presented in this report in a single paragraph. The paragraph should provide a general background of the problem, methodology and results.

Acknowledgements

This work would have not been possible without the input and support of many people over the last two trimesters. We would like to express my gratitude to everyone who contributed to it in some way or other.

First, we would like to thank my academic advisors, ...

Our sincere gratitude goes to ...

We are also thankful to ...

Last but not the least, We owe to our family including our parents for their unconditional love and immense emotional support.

Publication List

[Optional] The main contributions of this research are either published or accepted or in preparation in journals and conferences as mentioned in the following list:

Journal Articles

1.

Conference Papers

1.

Additional Publications

Following is the list of relevant publications published in the course of the research that is not included in the thesis:

1.

Table of Contents

Table of Contents v

List of Figures vi

List of Tables vii

1 Introduction 1

1.1 Project Overview . . . 1
1.2 Motivation . . . 2
1.3 Objectives . . . 3
1.4 Methodology . . . 3
1.5 Project Outcome . . . 3
1.6 Organization of the Report . . . 4

2 Background 5

2.1 Preliminaries . . . 5
2.1.1 Antimicrobial Resistance (AMR) . . . 5
2.1.2 The CARD Database . . . 5
2.1.3 Large Language Models (LLMs) and RAG . . . 6
2.2 Literature Review . . . 6
2.2.1 Similar Applications . . . 6
2.2.2 Related Research . . . 6
2.3 Gap Analysis . . . 7
2.4 Summary . . . 7

3 Project Design 8

3.1 Requirements Analysis . . . 8
3.1.1 Functional and Nonfunctional Requirements . . . 8
3.1.2 Context Diagram . . . 9
3.1.3 Data Flow Diagram Level 1 . . . 10
3.1.4 UI Design . . . 11
3.2 Detailed Methodology and Design . . . 12
3.3 Project Plan . . . 12

3.4 Task Allocation . . . 13
3.5 Summary . . . 14

4 Implementation and Results **15**
4.1 Environment Setup . . . 15
4.2 Testing and Evaluation . . . 15
4.3 Results and Discussion . . . 15
4.4 Summary . . . 15

5 Standards and Design Constraints **16**
5.1 Compliance with the Standards . . . 16
5.1.1 Software Standards . . . 16
5.1.2 Hardware Standards . . . 16
5.1.3 Communication Standards . . . 17
5.2 Design Constraints . . . 17
5.2.1 Economic Constraint . . . 17
5.2.2 Environmental Constraint . . . 17
5.2.3 Ethical Constraint . . . 17
5.2.4 Health and Safety Constraint . . . 17
5.2.5 Social Constraint . . . 17
5.2.6 Political Constraint . . . 17
5.2.7 Sustainability . . . 17
5.3 Cost Analysis . . . 17
5.3.1 Primary Budget . . . 17
5.3.2 Alternative Budget . . . 18
5.4 Complex Engineering Problem . . . 19
5.4.1 Complex Problem Solving . . . 19
5.4.2 Engineering Activities . . . 19
5.5 Summary . . . 20

6 Conclusion **21**
6.1 Summary . . . 21
6.2 Limitation . . . 21
6.3 Future Work . . . 21

References **24**

List of Figures

1.1 Overview flowchart of the proposed LLM-Driven framework for antimicrobial resistance gene detection and validation 2

3.1 Context diagram of the LLM-Driven ARG Framework showing interactions with external entities and internal processing components 10
3.2 Data Flow Diagram (Level 1) of the LLM-Driven ARG Framework illustrating internal modules and data movement 10
3.3 Proposed web-based user interface of the LLM-Driven ARG Framework showing file upload, processing status, and result visualization 12
3.4 A high level system architecture of the LLM-Driven framework for antimicrobial resistance gene detection and validation 12
3.5 Project timeline showing phase-wise distribution of tasks across three trimesters 13

List of Tables

5.1 Mapping with complex problem solving. 19
5.2 Mapping with complex engineering activities. 20

Chapter 1

Introduction

This chapter gives a general idea of our project. It covers why we chose this topic, what we want to achieve, a short summary of how we will do it, and what results we expect to see at the end.

1.1 Project Overview

Antimicrobial resistance (AMR) poses a major threat to human health around the world [1][2].The global burden of bacterial AMR compromises decades of clinical progress, as the rapid emergence of antibiotic resistance genes (ARGs) correlates directly with an excess risk of death and prolonged hospital stays [2].

Current bioinformatics tools for ARG annotation primarily rely on mathematical sequence alignment algorithms. While fast, standard alignment mechanisms often struggle to decipher borderline sequence variations, yielding false positive predictions or missed annotations.[3].

To address these limitations, we propose an LLM-driven[4] framework. Rather than relying strictly on direct sequence matching, our architecture takes a Genomic FASTA file as input and implements Retrieval-Augmented Generation (RAG)[5]. By integrating standard alignment tools such as DIAMOND[6] or BLAST[7] with the sequence modeling capabilities of a Large Language Model (LLM)[4] and the Comprehensive Antibiotic Resistance Database (CARD)[8], we aim to validate uncertain sequence matches. The framework extends beyond basic ARG annotation; it combines pre-trained parametric memory with explicit non-parametric memory to explain resistance mechanisms, generating human-readable summaries alongside structured JSON outputs for downstream genomic analysis.
Figure 3.4 presents a high-level overview of our proposed systems workflow, from FASTA input to validated resistance gene interpretation.

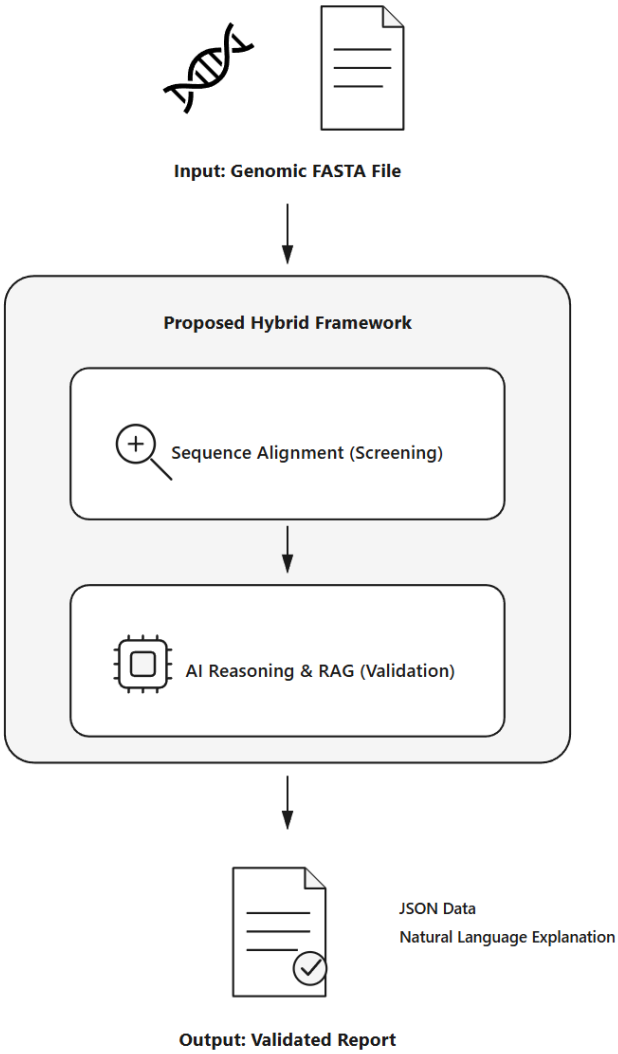

Figure 1.1: Overview flowchart of the proposed LLM-Driven framework for antimicrobial resistance gene detection and validation

1.2 Motivation

We decided to work on this project mainly because of the limitation of existing bioinformatics tools. Most of the current tools outputs a binary answer like "yes" or "no" based on mathematical scores. They do not understand the logic of the answer. If a result is ambiguous, a human researcher have to manually check databases, which takes a lot of time.

We wanted to build a system which acts like an expert assistant. We realized that we need to build a tool that can:

- Handle ambiguous cases.

- Collect evidence from different sources to make a decision.
- Explain the logic of resistance in simple words, not just codes.

By bulding the system, we can help researchers identify dangerous bacteria faster and more accurately, and reduce the chance of wrong diagnoses.

1.3 Objectives

The goal of our thesis project is to create a system that integrates the alignment of gene sequences with an AI reasoning system. Our objectives are as follows:

1. To develop a system for generating candidate ARG hits from Genome FASTA files by using DIAMOND[6], or BLAST[7] for aligning gene sequences.
2. To design a RAG (Retrieval-Augmented Generation) system that will use the CARD description database to create the logic and generate an answer.
3. To resolve ambiguities in matching that could be generated by the standard tools.
4. To decrease false positive results based on logical reasoning.
5. To generate output files that can contain both unstructured data (natural language summaries) and structured data such as JSON files that describe drug impact and mechanisms of resistance.

1.4 Methodology

There are several key elements of our process. The first element is taking raw genetic data and running it through a computational tool such as BLAST or DIAMOND to identify candidate genes. From these candidate genes, we do not automatically accept them. We instead use the RAG method. In the RAG method, the system searches the CARD ontology database for specific information about each gene on the list. The information gathered from the CARD ontology is then inputted into a large language model. The primary purpose of the large language model is to validate the output generated by the tool. Specifically, the large language model compares the alignment score produced by the tool with the biologic description of the genes and identifies whether the output generated by the tool accurately represents a real gene. Finally, the system produces a written report detailing its findings.

1.5 Project Outcome

At the end of this project, we expect to have a fully working software framework. The main outcomes will be:

- A system that produces minimum false flags compared to standard tools.
- Automatically generate reports that explains the resistance mechanisms and the drug impacts in natural language.
- A tool that can predict missing ARGs that other tools might overlooked.
- Multiple output formats (JSON + Text) that are useful for both developers and lab researchers.

1.6 Organization of the Report

The rest of this report is organized as follows: **Chapter 2** provides the background study and review of the literature. **Chapter 3** explains the methodology and system architecture. **Chapter 4** described the implementation and testing of the framework. Finally, **Chapter 5** concludes the report and possible future work.

Chapter 2

Background

To understand the rest of the report, we provided background information in this chapter . There are discussions of the basics of Antimicrobial Resistance (AMR), the databases we choose, and the current state of research in Large Language Models (LLMs) for bioinformatics.

2.1 Preliminaries

In this section, we cover the core concepts needed to understand our project: the threat of AMR, the CARD database, and the concept of RAG [9].

2.1.1 Antimicrobial Resistance (AMR)

When bacteria and/or viruses have developed resistance over time against certain drugs, they are unable to be treated by the drug (or drugs). The amount of time that an infection remains contagious increases due to this resistance. The "Global burden of bacterial antimicrobial resistance in 2019" study states that AMR is one of the top causes of death world-wide with millions of people dying each year from it[2]. Understanding which genetic mutations are involved in this type of resistance will help to protect public health.

2.1.2 The CARD Database

We will use the Comprehensive Antibiotic Resistance Database (CARD) for our project. CARD is an extensively utilized, curated repository of antibiotic resistance genes. The 2023 CARD release focuses on enabling machine learning models to predict the resistome [8]. CARD organizes its data using the Antibiotic Resistance Ontology (ARO), which facilitates the computer's ability to understand how a particular gene relates to the antibiotics that it confers resistance against.

2.1.3 Large Language Models (LLMs) and RAG

Retrieval-Augmented Generation (RAG) is a method used to reduce errors caused by hallucinations in LLMs when using large datasets in areas like biology such as genetic coding [5] . This is accomplished by allowing the LLM to search for relevant and trusted information from an external source such as CARD prior to generating its final answer, providing increased accuracy compared to generated answers that were based upon no retrieval at all.

2.2 Literature Review

This section reviews existing tools and recent research papers related to our work.

2.2.1 Similar Applications

Currently, most researchers use traditional methods based on sequence alignments in order to discover the presence of known resistance genes.

- **Alignment Methods:** Most commonly used methods include, for example, BLAST (Basic Local Alignment Search Tool) [7] and DIAMOND [6]. Both of these methods rely on comparing an unknown DNA sequence with a large database of known sequences to determine if there is any similarity between them. The problem with both of these methods is that they are very restrictive. Each has its own strict mathematical criteria that each comparison must meet in order to be identified as a valid match. Any deviation in this comparison results in the method either failing to identify the match or identifying it incorrectly.

- **Predictive Models:** More recently, researchers have been developing predictive models using evolutionary algorithms along with "Mixture of Experts" to predict the presence of resistance genes [1]. Although these models provide a significant improvement over traditional alignment methods by providing a means to generate human readable explanations regarding the predicted presence of resistance genes; their primary function remains to make predictions of whether or not a given gene will exhibit resistant characteristics.

2.2.2 Related Research

Transformers and LLMs have become increasingly used to process genomic data over the past few years[4]. The use of transformers and LLMs to process genomic data includes:

- **Genomic foundation models**: DNABERT [10], DNABERT-2 [11] and HyenaDNA [12] have demonstrated that DNA can be treated as a language. The models treat k-mers of DNA as if they were words in a language and therefore can learn patterns in DNA sequences; these are useful for classification but typically don't provide an explanation as to why a sequence is resistant to drugs.

- **Bio-medical text models**: BioGPT [13] is a model specifically trained on biomedical literature. BioGPT is capable of producing text on biology but was not intended to analyze raw FASTA files directly for drug resistance determination.
- **LLM visualization**: AuraGenome [14] demonstrates how LLMs may also assist in visualizing circular genomes; this further supports the increasing use of LLMs to make large amounts of complex biological data "human-friendly," which we believe will support our goal of providing users with readable reports.

2.3 Gap Analysis

Based on the literature review, we can see clearly where our project fits into the gaps that exist within the current body of knowledge:

1. **Lack of reasoning in alignment tools:** All standard alignment tools (BLAST[7], DIAMOND[6]) provide a score and nothing else. The tools are unable to reason about why a particular score has been provided. If there is a borderline case based upon the degree of sequence similarity, they are unable to tell you whether the mutations are safe or dangerous.
2. **The black box nature of genomic AI models:** Genomic models such as DNABERT[10] have the potential to be very powerful; however, these models operate like a black box. The model will provide a classification and it does not provide any indication of the biological mechanism(s) which lead to the classification.
3. **The problem of the false positive:** All previous methods have struggled with false positives when dealing with ambiguous cases. To date, there is no existing method/framework that provides both the speed of sequence alignment and the **reasoning capability** of an LLM to cross reference ontology context to double-check the borderline hits.

Our proposed framework addresses all three of the above issues. We utilize the RAG module to integrate evidence from multiple sources and provide a natural language explanation for each of the decisions made.

2.4 Summary

In this chapter, we reviewed the serious threat of AMR and the tools currently used to fight it. We looked at the CARD database and recent advances in Genomic LLMs like DNABERT and BioGPT. While these tools are powerful, we found that a system capable of explaining its results and validating ambiguous hits using reasoning is missing. The next chapter will describe how we built our system to solve this problem.

Chapter 3

Project Design

This chapter describes the overall strategy for the entire project as well as the methodology we used to create our proposed framework. In this chapter, we will outline the general needs of our proposed system; we will show the system flow by means of a diagram; we will describe the specific technology decisions we made; and finally, we will provide a timeline of how we completed the project.

3.1 Requirements Analysis

We had to determine before writing any code what was missing from existing bioinformatics applications and define exactly what our system would need to do. Our primary objective is to develop an expert assistant to confirm "borderline" resistance gene matches.

3.1.1 Functional and Nonfunctional Requirements

Functional Requirements

The primary functionalities of the system are to be able to satisfy the objectives defined in Chapter 1 and these are:

- **Input Processing:**The system must accept raw genomic data in either FASTA format or plain text format as its source of initial data.
- **Sequence Alignment:**The system must include common sequence alignment tools (such as DIAMOND[6] and BLAST[7]), which generate a set of candidate ARGs[2] based on the alignments generated during the alignment process.
- **Context Retrieval (RAG[5]):** For each of the candidate ARGs previously identified by the sequence alignment step, the system must retrieve all relevant resistance mechanisms and drug interactions for each of those candidate genes using the CARD database[8].

- **LLM Validation:** The system must then send an alignment score and CARD[8] description of the gene to an LLM. The LLM will then validate whether the identified gene was a true positive or false positive using this information.
- **Dual-Format Output:** The system must produce results in at least two different formats. The first is a structured JSON file, which can be used by developers. The second is a natural language summary of the resistance mechanism of the identified gene, which can be used by researchers.

Non-functional Requirements

Non-functional requirements describe how well the system will work in terms of quality and performance.

- **Accuracy:** The accuracy of this system should be higher than that of using BLAST or DIAMOND because it has lower rates of false positives and false negatives.
- **Explainability:** This system should provide a natural language explanation for why a gene was classified as drug-resistant; whereas, the AI-based systems are typically "black box" type models.
- **Modularity:** Each module should be developed using a microservices approach, so that if you wanted to swap out the alignment tool or the LLM backend (e.g., GPT-4 with a local LLaMA model), you could do that without affecting the other modules' functionality.

3.1.2 Context Diagram

To see where our system fits in the broader world picture we developed a Context Diagram to show this. At the center of the diagram is the "LLM-Driven ARG Framework" which is the main process for the system. The Primary User will provide the FASTA file. There are two Major External Entities that the system will interact with: CARD Database (to get ontology data); and LLM Provider (to reason/validate). The Sequence Alignment Tool serves as an internal Processing Engine to feed Data into the Main Logic of the System.

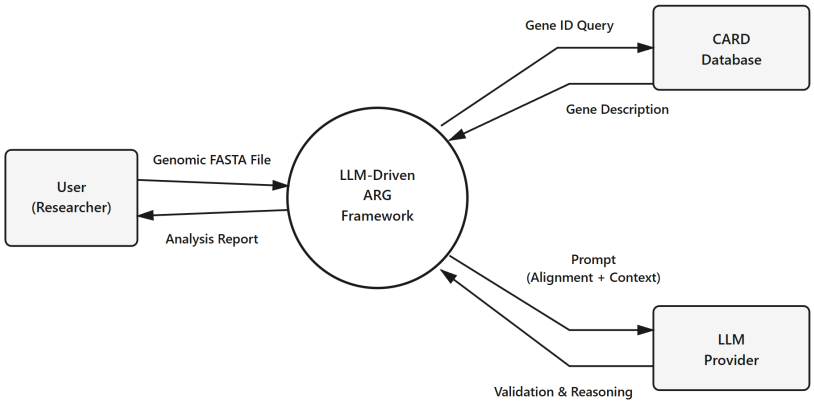

Figure 3.1: Context diagram of the LLM-Driven ARG Framework showing interactions with external entities and internal processing components

3.1.3 Data Flow Diagram Level 1

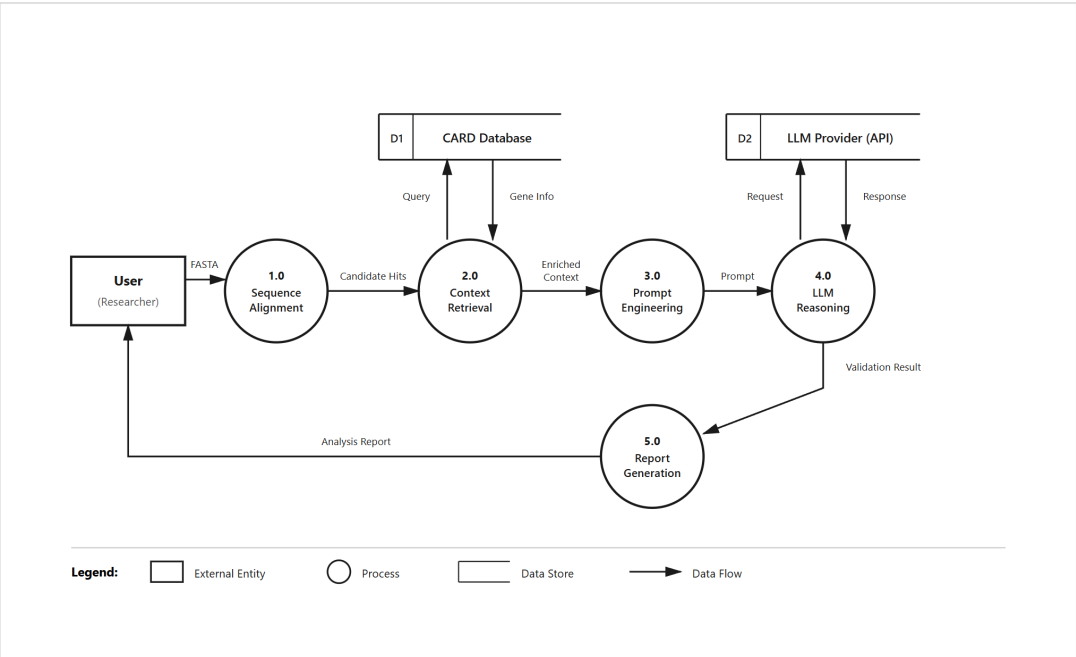

Figure 3.2: Data Flow Diagram (Level 1) of the LLM-Driven ARG Framework illustrating internal modules and data movement

We used a DFD Level-1 to represent the flow of data through the individual components in our overall system by expanding upon the context diagram developed earlier.

- **Data Ingestion**: The user uploads a FASTA file to the system.

- **Alignment Process**: The uploaded file is then sent to the Alignment Module and it will produce an ordered list of candidate genes that were identified from the uploaded file along with each corresponding alignment score.
- **Knowledge Retrieval**: The RAG Module takes the candidate gene identifier(s), identifies the relevant ontology entry(s) using the CARD Ontology[8], and retrieves biological description(s) of the candidate gene(s), as well as the drug class(es) and/or mechanism(s) of resistance.
- **Prompt Engineering**: The Context Builder uses the raw statistical alignment values as input combined with the previously obtained biological knowledge and generates a structured query or prompt based on this input.
- **Reasoning Engine**: The LLM[4] receives the structured prompt as input, performs analysis of the content of the prompt to determine whether the candidate gene is valid, and returns a determination of validity along with an explanation of why such a determination was made.
- **Report Generation**: The last component in the system processes the validation report into a format suitable for both machine consumption as structured JSON and provides a readable summary of the validation results to the end-user.

3.1.4 UI Design

We wanted our tool to have an easy-to-use interface, so that researchers without extensive experience with coding can easily use it. For this reason, we opted for a Web-Based Dashboard as the UI of our Tool.

Below are the features of our dashboard:

- **Upload Section**: The area where users can upload their FASTA files by dragging them onto the zone or selecting the file from the directory.
- **Status Panel:** A progress bar, that shows the status of the three processes (Alignment -¿ Retrieval -¿ Reasoning) in real time.
- **Results View:** Two areas on screen, that show two different views of the results. The right side will display a table of the genes found, and when you select a gene from the list, the lower side of the screen will display both the LLM's natural language explanation of the selected gene and the level of confidence associated with the selected gene.

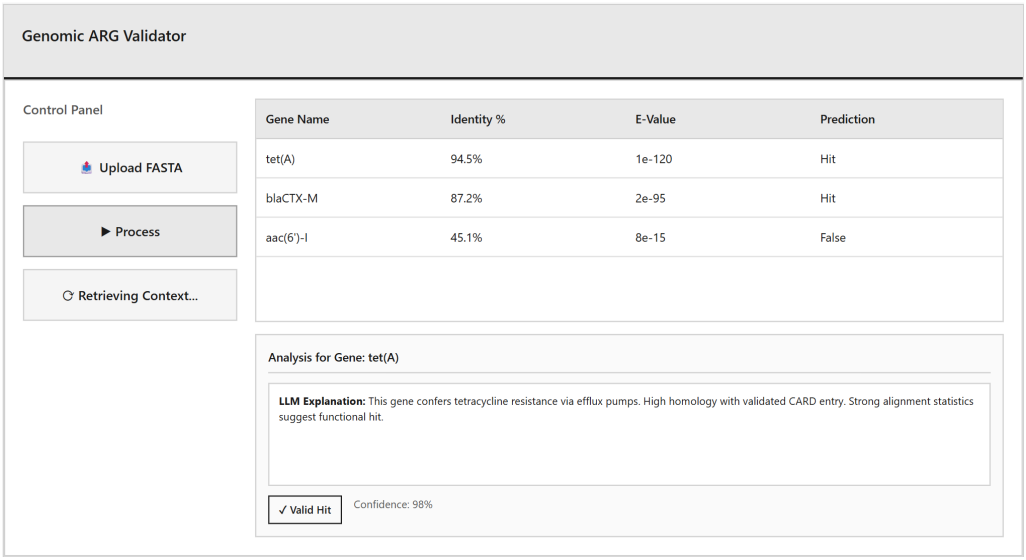

Figure 3.3: Proposed web-based user interface of the LLM-Driven ARG Framework showing file upload, processing status, and result visualization

3.2 Detailed Methodology and Design

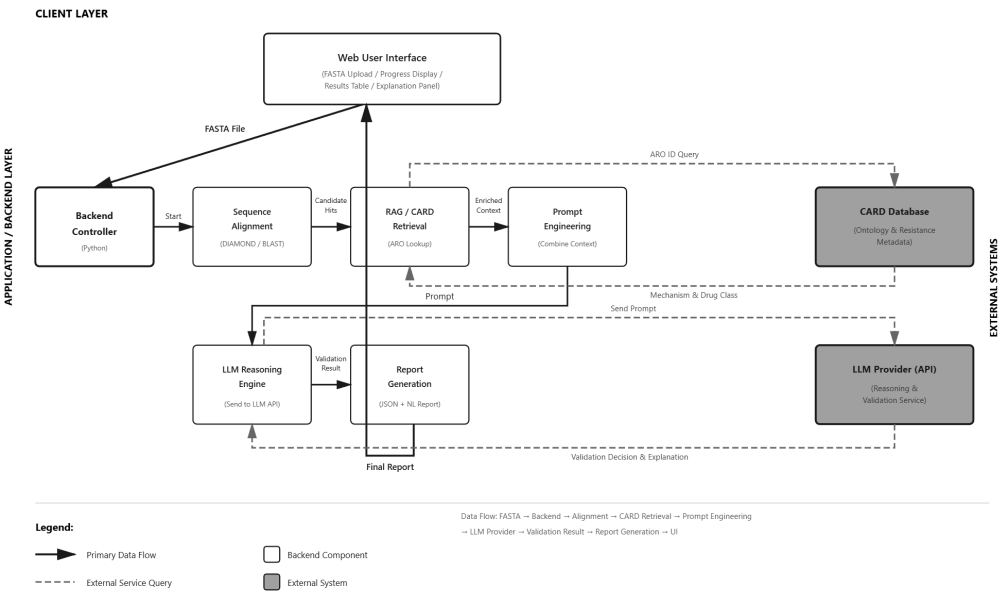

Figure 3.4: A high level system architecture of the LLM-Driven framework for antimicrobial resistance gene detection and validation

3.3 Project Plan

We had limited time to finish this project. To ensure we completed the project on time, we divided our work into multiple phases over the three trimesters.

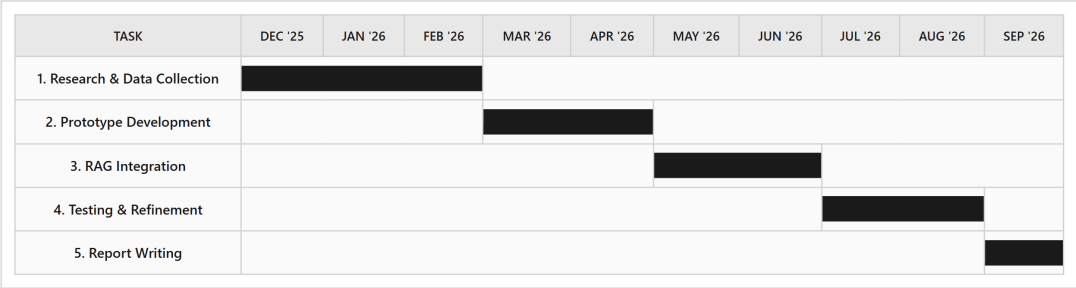

Figure 3.5: Project timeline showing phase-wise distribution of tasks across three trimesters

The full project was divided into the following phases:

- **Phase 1: Research and Data Collection (Months 1 – 3)** We started off by reviewing the CARD database and learning about how FASTA files operate. In addition, we analyzed previous research related to Genomic Foundation Models and RAG systems.
- **Phase 2: Development of a Prototype (Months 4 – 5)** Next, we developed a basic Python script that would execute the local alignment tools and analyze their outputs. Then, we linked the LLM API to check whether the LLM could actually interpret the biological descriptions.
- **Phase 3: Linking to RAG Ontology (Month 6 – 7)** Then, we linked the outputs of the alignment tools to the CARD ontology. This was the most difficult phase of our project because we needed to ensure that the correct description was retrieved for each hit.
- **Phase 4: Testing and Refining the System (Month 8 – 9)** Afterwards, we tested our system with several different datasets that were already identified to determine whether or not the system was able to successfully identify resistance genes in them. Next, we adjusted the "prompts" provided to the LLM so that it would not provide generic responses.
- **Phase 5: Write Up Results (Month 10)** Finally, we documented the results of our project, produced the tables comparing our results, and prepared this thesis document.

3.4 Task Allocation

In order to divide the work load fairly among the five members of our team because of the multi-components nature of this project (biological data processing, artificial intelligence and web development), each member worked on specific areas of the project rather than all members contributing to the entire project. Each member used their unique skills and abilities as well as collaborated on the overall research of the project and debugging efforts.

Backend and LLM Integration

The primary "brain" of our framework was built by **Abdullah Al Noman**, and **Md. Habibulla Misba**. The primary responsibilities of these two team members was to establish the python environment and to integrate the sequence alignment tools (DIAMOND/BLAST) into the project. Additionally, they were also responsible for creating an interface with the LLM API which included writing the logic for developing the prompts to pass the proper context to the LLM to generate a response.

RAG System and Database Management

Kazi Neyamul Hasan was responsible for the "retrieval" portion of our RAG architecture. Neyamul studied the CARD database to develop a process to properly retrieve the resistance mechanism descriptions based on the gene ID's returned by the alignment tools. This provided the LLM with consistent and accurate biological information to use when generating responses.

Front-end Development and User Interface

To provide a user-friendly experience for researchers, **Rakibul Hassan** was responsible for the User Interface of the system. Rakib designed and developed the web-based dashboard through which users could upload fasta files and display the output of the system. Rakib ensured that the complex json output from the backend was presented to the user in a clean, easy-to-read format.

Testing, Validation, and Documentation

Mahathir Mohammad was responsible for the testing and validation of the system and the documentation of the project. Mahathir tested the system using multiple test datasets to verify that the system produced no false positives or other types of errors. Additionally, Mahathir documented the project including the compilation of the research findings and ensured that the final thesis document complied with the universities' guidelines for format.

The following table illustrates the division of responsibilities among the team:

Student ID	Name	Primary Responsibilities
0112230367	Abdullah Al Noman	Project Lead, Backend Logic, LLM Integration
011221373	Md. Habibulla Misba	Sequence Alignment Pipeline, Python Scripting
0112230359	Kazi Neyamul Hasan	CARD Database Integration, RAG Implementation
0112230362	Rakibul Hassan	Frontend Design, UI/UX, Data Visualization
0112230889	Mahathir Mohammad	Software Testing, Dataset Validation, Report Writing

3.5 Summary

Chapter 4

Implementation and Results

[Must be present in Final Report. Incomplete version might be included in FYDP-1 Report, however it is optional.]

Every chapter should start with 1-2 sentences on the outline of the chapter.

4.1 Environment Setup

4.2 Testing and Evaluation

4.3 Results and Discussion

4.4 Summary

Chapter 5

Standards and Design Constraints

[Must be present in FYDP-1 Report and also in Final Report]

Every chapter should start with 1-2 sentences on the outline of the chapter.

5.1 Compliance with the Standards

Only mention the standards that are related to your project. This list is not complete. For each of the standards discuss the alternates with pros and cons and rationale of selection.

5.1.1 Software Standards

The system is largely dependent on backend processing of the large volumes of biological data it receives. Because we have written the core alignment pipeline and all of the API logic in Python, we followed the guidelines defined by PEP-8 throughout our entire codebase. The framework also restricts the input data formats to FASTA for sequencing data (thus excluding the added overhead of processing Fastq quality scores), which significantly optimized the data engineering workflow. The backend controller provides the output as JSON files along with a text summary generated from a Large Language Model (LLM). We chose JSON instead of XML as an output file type because JSON is easier to parse than XML in web-based dashboard interfaces where the data is visualized and therefore will make it easier for the user to visualize the data via the frontend interface.

5.1.2 Hardware Standards

Our initial hardware requirements are determined by executing the alignment tools locally. The tool assumes an x86-64 CPU based system. Many legacy bioinformatics tools that have typical configurations for BLAST and DIAMOND do not natively run on ARM architectures. While it would be possible to emulate x86 on an ARM processor, this emulation severely impacts the time required to perform the sequence alignment step. If we consider that a user can choose to replace the external LLM provider with a local model (for example, a local LLAMA model) then our hardware standards will shift in order to require

a dedicated CUDA compatible GPU. In order to keep our initial hardware requirements low enough to allow operation on most standard clinical computers, we default to use cloud-based LLM providers.

5.1.3 Communication Standards

Data is exchanged with external components as well as with the application back-end by means of https/rest interfaces. The RAG Module sends http-requests to the CARD-Database to retrieve Gene Ontology information. In addition, the sending of structured input to the LLM Reasoning Engine uses secure API Endpoints. Since we did not need WebSockets for the validation of Candidate Hits as this is an isolated (request-response) Event instead of a continuous Data Stream. By utilizing REST, we reduced the State Management Load of our Back-End Servers.

5.2 Design Constraints

Only mention the constraints that are related to the design of your project. This list is not complete.

5.2.1 Economic Constraint

5.2.2 Environmental Constraint

5.2.3 Ethical Constraint

5.2.4 Health and Safety Constraint

5.2.5 Social Constraint

5.2.6 Political Constraint

5.2.7 Sustainability

5.3 Cost Analysis

This section describes the financial investment needed for the implementation of the framework, as well as presents a potential commercial sustainability model. The financial estimates are based upon a rough estimate of 1 USD to 120 BDT.

5.3.1 Primary Budget

The primary budget is designed to use remote cloud hosting in conjunction with "pay as you go" external large language model (LLM) APIs to reduce the initial capital expenditure on the system.

Hosting:

To host the web dashboard, python backend and local alignment algorithms (DIAMOND & BLAST), a cloud virtual private server (VPS) is required. The sequence alignment process will require that all reference database are loaded into memory, therefore a server with at least 4gb of ram and 2vCPUs is the minimum specification requirement. With the current standard pricing of 2026 for service providers such as DigitalOcean and Vultr, a 4GB server is priced at approximately $24.00 per month, which is equivalent to approximately 2880 BDT.

API Usage:

The reasoning engine will send alignment scores and CARD descriptions to an external LLM using the api. As of the time of this writing, the cost of using models such as GPT-4O-Mini, where the cost is $0.15 per million input tokens and $0.60 per million output tokens, will be approximately $3.00 to process a standard academic volume of 100 Fasta File Queries per month. This will add approximately 360 bdt to the monthly budget.

Estimated Total Cost:

The combined monthly operational cost to reliably operate the framework is estimated to be approximately 3240 bdt.

5.3.2 Alternative Budget

An alternative to using cloud computing would be to buy a workstation that can run a local version of the open-source LLM and perform sequence alignment. As a result, both a genomic database and an AI-based reasoning engine require considerable amounts of memory and graphics processing. Using prices from 2026 from local Bangladeshi IT hardware retailers for their products, the estimated costs for a workstation with the appropriate components are as follows:

- **GPU (NVIDIA RTX 4090 24GB):** In order to run the LLM in a smooth manner, it is necessary to have a GPU with at least 24 GB of memory. Local Bangladeshi retailers currently charge anywhere from 275,000 to 300,000 BDT depending upon the manufacturer.
- **CPU (AMD Ryzen 9 7900X):** To support the use of standard alignment tools, which are multi-threaded applications, it is necessary to have a CPU that supports multiple threads. The cost of this CPU is approximately 43,500 BDT.
- **Memory & Storage (64 GB DDR5; 2 TB NVMe Gen4):** It is also essential to load the CARD database into memory along with loading the raw genetic file into memory when performing sequence alignment. Therefore, the combined cost of memory & storage is approximately 45,000 BDT.

- **Additional Hardware:** The additional hardware requirements include a 1000W + power supply, compatible motherboard, cooler, and case, all of which will cost approximately 70,000 BDT.
- **Estimated Total Cost:** The estimated total cost for building a workstation with enough capabilities to be used as a local server is approximately 433,500 to 458,500 BDT.

Why We Chose the Primary Budget

We chose the primary budget because spending over 400,000 Taka for a high-end workstation for an academic prototype is simply impossible. In fact, the hardware purchase model only makes economic sense for very large scale enterprise laboratories. Those types of laboratories process many thousands of genomic sequences every day, and they are typically required by law to store patient information on premises rather than sending it to an external API to prevent the potential loss or misuse of sensitive medical information.

5.4 Complex Engineering Problem

5.4.1 Complex Problem Solving

In this section, provide a mapping with problem solving categories. For each mapping add subsections to put rationale (Use Table 5.1). For P1, you need to put another mapping with Knowledge profile and rational thereof.

Table 5.1: Mapping with complex problem solving.

P1 Dept of Knowledge	P2 Range of Conflicting Requirements	P3 Depth of Analysis	P4 Familiarity of Issues	P5 Extent of Applicable Codes	P6 Extent of Stakeholder Involvement	P7 Interdependence
√	√					

5.4.2 Engineering Activities

In this section, provide a mapping with engineering activities. For each mapping add subsections to put rationale (Use Table 5.2).

Table 5.2: Mapping with complex engineering activities.

A1 Range of resources	A2 Level of Interaction	A3 Innovation	A4 Consequences for society and environment	A5 Familiarity
√	√			

5.5 Summary

Chapter 6

Conclusion

[Must be present in FYDP-1 Report and also in Final Report. Might be incomplete in FYDP-1 Report.]

Every chapter should start with 1-2 sentences on the outline of the chapter.

6.1 Summary

6.2 Limitation

6.3 Future Work

References

[1] Anshul Bagaria. AMR-MoEGA: Antimicrobial Resistance Prediction using Mixture of Experts and Genetic Algorithms, November 2025.

[2] Christopher J. L. Murray, Kevin Shunji Ikuta, Fablina Sharara, Lucien Swetschinski, Gisela Robles Aguilar, Authia Gray, Chieh Han, Catherine Bisignano, Puja Rao, Eve Wool, Sarah C. Johnson, Annie J. Browne, Michael Give Chipeta, Frederick Fell, Sean Hackett, Georgina Haines-Woodhouse, Bahar H. Kashef Hamadani, Emmanuelle A. P. Kumaran, Barney McManigal, Sureeruk Achalapong, Ramesh Agarwal, Samuel Akech, Samuel Albertson, John Amuasi, Jason Andrews, Aleskandr Aravkin, Elizabeth Ashley, François-Xavier Babin, Freddie Bailey, Stephen Baker, Buddha Basnyat, Adrie Bekker, Rose Bender, James A. Berkley, Adhisivam Bethou, Julia Bielicki, Suppawat Boonkasidecha, James Bukosia, Cristina Carvalheiro, Carlos Castañeda-Orjuela, Vilada Chansamouth, Suman Chaurasia, Sara Chiurchiù, Fazle Chowdhury, Rafai Clotaire Donatien, Aislinn J. Cook, Ben Cooper, Tim R. Cressey, Elia Criollo-Mora, Matthew Cunningham, Saffiatou Darboe, Nicholas P. J. Day, Maia De Luca, Klara Dokova, Angela Dramowski, Susanna J. Dunachie, Thuy Duong Bich, Tim Eckmanns, Daniel Eibach, Amir Emami, Nicholas Feasey, Natasha Fisher-Pearson, Karen Forrest, Coralith Garcia, Denise Garrett, Petra Gastmeier, Ababi Zergaw Giref, Rachel Claire Greer, Vikas Gupta, Sebastian Haller, Andrea Haselbeck, Simon I. Hay, Marianne Holm, Susan Hopkins, Yingfen Hsia, Kenneth C. Iregbu, Jan Jacobs, Daniel Jarovsky, Fatemeh Javanmardi, Adam W. J. Jenney, Meera Khorana, Suwimon Khusuwan, Niranjan Kissoon, Elsa Kobeissi, Tomislav Kostyanev, Fiorella Krapp, Ralf Krumkamp, Ajay Kumar, Hmwe Hmwe Kyu, Cherry Lim, Kruy Lim, Direk Limmathurotsakul, Michael James Loftus, Miles Lunn, Jianing Ma, Anand Manoharan, Florian Marks, Jürgen May, Mayfong Mayxay, Neema Mturi, Tatiana Munera-Huertas, Patrick Musicha, Lilian A. Musila, Marisa Marcia Mussi-Pinhata, Ravi Narayan Naidu, Tomoka Nakamura, Ruchi Nanavati, Sushma Nangia, Paul Newton, Chanpheaktra Ngoun, Amanda Novotney, Davis Nwakanma, Christina W. Obiero, Theresa J. Ochoa, Antonio Olivas-Martinez, Piero Olliaro, Ednah Ooko, Edgar Ortiz-Brizuela, Pradthana Ounchanum, Gideok D. Pak, Jose Luis Paredes, Anton Yariv Peleg, Carlo Perrone, Thong Phe, Koukeo Phommasone, Nishad Plakkal, Alfredo Ponce-de Leon, Mathieu Raad, Tanusha Ramdin, Sayaphet Rattanavong, Amy Riddell, Tamalee Roberts, Julie Victoria Robotham, Anna Roca, Victor Daniel

Rosenthal, Kristina E. Rudd, Neal Russell, Helio S. Sader, Weerawut Saengchan, Jesse Schnall, John Anthony Gerard Scott, Samroeng Seekaew, Mike Sharland, Madhusudhan Shivamallappa, Jose Sifuentes-Osornio, Andrew J. Simpson, Nicolas Steenkeste, Andrew James Stewardson, Temenuga Stoeva, Nidanuch Tasak, Areerat Thaiprakong, Guy Thwaites, Caroline Tigoi, Claudia Turner, Paul Turner, H. Rogier van Doorn, Sithembiso Velaphi, Avina Vongpradith, Manivanh Vongsouvath, Huong Vu, Timothy Walsh, Judd L. Walson, Seymour Waner, Tri Wangrangsimakul, Prapass Wannapinij, Teresa Wozniak, Tracey E. M. W. Young Sharma, Kalvin C. Yu, Peng Zheng, Benn Sartorius, Alan D. Lopez, Andy Stergachis, Catrin Moore, Christiane Dolecek, and Mohsen Naghavi. Global burden of bacterial antimicrobial resistance in 2019: a systematic analysis. *The Lancet*, 399(10325):629–655, February 2022. Publisher: Elsevier.

[3] Hyunwoo Yoo, Bahrad Sokhansanj, James R. Brown, and Gail Rosen. Predicting Anti-microbial Resistance using Large Language Models, January 2024. arXiv:2401.00642 [cs].

[4] Zhenyu Wang, Zikang Wang, Jiyue Jiang, Pengan Chen, Xiangyu Shi, and Yu Li. Large Language Models in Bioinformatics: A Survey, May 2025. arXiv:2503.04490 [cs].

[5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, May 2020.

[6] Benjamin Buchfink, Chao Xie, and Daniel H. Huson. Fast and sensitive protein alignment using DIAMOND. *Nature Methods*, 12(1):59–60, January 2015.

[7] S. F. Altschul, W. Gish, W. Miller, E. W. Myers, and D. J. Lipman. Basic local alignment search tool. *Journal of Molecular Biology*, 215(3):403–410, October 1990.

[8] Brian P. Alcock, William Huynh, Romeo Chalil, Keaton W. Smith, Amogelang R. Raphenya, Mateusz A. Wlodarski, Arman Edalatmand, Aaron Petkau, Sohaib A. Syed, Kara K. Tsang, Sheridan J. C. Baker, Mugdha Dave, Madeline C. McCarthy, Karyn M. Mukiri, Jalees A. Nasir, Bahar Golbon, Hamna Imtiaz, Xingjian Jiang, Komal Kaur, Megan Kwong, Zi Cheng Liang, Keyu C. Niu, Prabakar Shan, Jasmine Y. J. Yang, Kristen L. Gray, Gemma R. Hoad, Baofeng Jia, Timsy Bhando, Lindsey A. Carfrae, Maya A. Farha, Shawn French, Rodion Gordzevich, Kenneth Rachwalski, Megan M. Tu, Emily Bordeleau, Damion Dooley, Emma Griffiths, Haley L. Zubyk, Eric D. Brown, Finlay Maguire, Robert G. Beiko, William W. L. Hsiao, Fiona S. L. Brinkman, Gary Van Domselaar, and Andrew G. McArthur. CARD 2023: expanded curation, support for machine learning, and resistome prediction at the Comprehen-

sive Antibiotic Resistance Database. *Nucleic Acids Research*, 51(D1):D690–D699, January 2023.

[9] Jon Kleinberg and Eva Tardos. *Algorithm design*. Pearson Education India, 2006.

[10] Yanrong Ji, Zhihan Zhou, Han Liu, and Ramana V Davuluri. DNABERT: pre-trained Bidirectional Encoder Representations from Transformers model for DNA-language in genome. *Bioinformatics*, 37(15):2112–2120, August 2021.

[11] Zhihan Zhou, Yanrong Ji, Weijian Li, Pratik Dutta, Ramana Davuluri, and Han Liu. DNABERT-2: Efficient Foundation Model and Benchmark For Multi-Species Genome, March 2024. arXiv:2306.15006 [q-bio].

[12] Eric Nguyen, Michael Poli, Marjan Faizi, Armin Thomas, Callum Birch-Sykes, Michael Wornow, Aman Patel, Clayton Rabideau, Stefano Massaroli, Yoshua Bengio, Stefano Ermon, Stephen A. Baccus, and Chris Ré. HyenaDNA: Long-Range Genomic Sequence Modeling at Single Nucleotide Resolution, June 2023.

[13] Renqian Luo, Liai Sun, Yingce Xia, Tao Qin, Sheng Zhang, Hoifung Poon, and Tie-Yan Liu. BioGPT: Generative Pre-trained Transformer for Biomedical Text Generation and Mining, October 2022.

[14] Chi Zhang, Yu Dong, Yang Wang, Yuetong Han, Guihua Shan, and Bixia Tang. AuraGenome: An LLM-Powered Framework for On-the-Fly Reusable and Scalable Circular Genome Visualizations. *IEEE Computer Graphics and Applications*, 45(5):78–92, September 2025.